TrustYojana: A Verifiable RAG Question-Answering System for Indian Government Welfare Schemes

Project Report · LLM End-Semester Evaluation (ESE)

Integrated application combining Document-based RAG (Experiment 5), LangChain/LangGraph Workflow (Experiment 6) and FastAPI + Streamlit Deployment (Experiment 7)

Submitted by: Hetanshi Shah · **Roll No. / Enrolment No.:** 2023300199 · **Class / Division:** CE Div C

Course: LLM Production and Deployment

Academic Year: 2026-27 · **Date:** Nov 3, 2026

Repository: https://github.com/hetanshi18/llm-ese-trustyojana

Index

No.	Section	Covers
1	Abstract	What TrustYojana is and what was built
2	Introduction	2.1 Problem statement; 2.2 Objectives; 2.3 Scope and schemes covered; 2.4 Technology stack; 2.5 Data sources; 2.6 Mapping to the lab experiments
3	Application Workflow	3.1 System architecture; 3.2 Document ingestion; 3.3 Retrieval; 3.4 LangGraph workflow (state, prompts, evidence check); 3.5 Personal eligibility; 3.6 REST API (request lifecycle, response, errors); 3.7 Streamlit interface; 3.8 Privacy, logging and traceability; 3.9 Configuration and LLM layer; 3.10 Setup and execution
4	Results and Observations	4.1 Test scenarios; 4.2 Citations, freshness and execution trace; 4.3 Failure handling; 4.4 Observations; 4.5 Design decisions
5	Conclusion	5.1 Summary; 5.2 Limitations; 5.3 Future scope

1. Abstract

TrustYojana answers questions about Indian government welfare schemes only when the retrieved official documents support the answer, and abstains otherwise. It is one integrated application that combines the three lab experiments: document-based RAG (Experiment 5), a LangGraph workflow (Experiment 6) and a FastAPI backend with a Streamlit interface (Experiment 7).

The knowledge base is five official PDFs covering four schemes: PM-KISAN, Ayushman Bharat PM-JAY, the National Means-cum-Merit Scholarship Scheme (NMMSS) and PM SVANidhi. The ingestion pipeline

extracts text with PyMuPDF, cleans it, splits it into 800-character chunks with 100 characters of overlap, embeds each chunk with BAAI/bge-small-en-v1.5 and stores it in a persistent ChromaDB collection. SHA-256 file hashes let repeat runs skip unchanged documents.

At question time, a seven-node LangGraph workflow classifies the intent and scheme, retrieves the top four chunks, drafts a grounded answer and checks every claim against the retrieved text with an LLM grader. The grader returns SUPPORTED, NOT_ENOUGH_EVIDENCE or CONTRADICTED. A failed check triggers a query rewrite and up to two retries before the system falls back to a safe abstention message. Personal eligibility questions get a stateless, criterion-by-criterion assessment (Met, Not met, Unknown) with a list of missing information and a mandatory disclaimer.

The workflow is served through a FastAPI backend with request tracking, health monitoring and privacy-preserving logs, and consumed by a Streamlit chat interface that shows verdict badges, page-level citations, freshness notices and an execution trace. The LLM layer is provider-agnostic (OpenAI, Gemini, Anthropic or Ollama); the committed configuration runs a local Ollama model, qwen2.5:0.5b.

Keywords: Retrieval-Augmented Generation, LangGraph, ChromaDB, claim-level verification, abstention, FastAPI, Streamlit, government schemes

2. Introduction

Citizens often cannot tell what a welfare scheme pays, who qualifies or which documents they need, and rules change over time. A general-purpose chatbot can answer fluently from memory, but a confident wrong answer about eligibility or benefit amounts can cost someone a real entitlement. TrustYojana is built around one rule: answer only what the official documents support, show where the answer came from, and say so plainly when it cannot answer.

2.1 Problem Statement

A plain retrieve-and-generate pipeline has no way to detect that its own draft goes beyond the retrieved text. For government schemes this is a real risk, because amounts, installments, exclusion rules and eligibility conditions are precise facts. The project therefore adds a verification step after generation and an abstention path when verification fails.

2.2 Objectives

- Build a document-based RAG pipeline over official scheme documents, with page-level source metadata.
- Verify every drafted answer claim by claim against the retrieved passages before showing it.
- Retry with a rewritten search query when evidence is missing or contradicted, then abstain safely if retries are exhausted.
- Handle personal eligibility questions without issuing a definitive decision, listing what is Met, Not met and Unknown.
- Expose the system as a validated REST API with health monitoring and privacy-preserving logs.
- Provide a chat interface that makes the evidence visible: verdict, citations, freshness and execution trace.
- Keep the LLM provider swappable through configuration.

2.3 Scope and Schemes Covered

The system covers four schemes: PM-KISAN, Ayushman Bharat PM-JAY, NMMSS Scholarship and PM SVANidhi. Questions about anything else are classified as out of scope and routed straight to a fallback message without touching the vector store. Each answer is a single-turn response; the app does not keep conversational memory on the server, and the Streamlit chat history exists only within the browser session.

2.4 Technology Stack

Layer	Technology
Language	Python 3.11
Document parsing	PyMuPDF for PDFs; BeautifulSoup for HTML; plain TXT and MD
Chunking	LangChain RecursiveCharacterTextSplitter (chunk size 800, overlap 100)
Embeddings	BAAI/bge-small-en-v1.5 through SentenceTransformers, on CPU
Vector store	ChromaDB, persistent, cosine similarity
Orchestration	LangChain and LangGraph StateGraph
LLM	Provider-agnostic: OpenAI, Gemini, Anthropic or Ollama; temperature 0. Configured model: qwen2.5:0.5b on Ollama
Backend	FastAPI with Uvicorn; Pydantic v2 models
Frontend	Streamlit chat interface
Configuration	config.yaml plus .env, loaded by a Pydantic config module

2.5 Data Sources

The knowledge base is five official PDFs in data/raw/, described in data/sources.csv. Each chunk carries the scheme, title, source file, page, official URL, document date and date accessed, which is what makes page-level citation and the freshness notice possible.

File	Scheme	Document	Document date	Official site
pm_kisan_guidelines.pdf	PM-KISAN	PM-KISAN Operational Guidelines (Revised 2020)	2020-03-29	pmkisan.gov.in
ayushman_pmjay_guidelines.pdf	Ayushman Bharat PM-JAY	AB PM-JAY Beneficiary Identification Guidelines	2020-01-15	pmjay.gov.in
nmmss_guidelines	NMMSS	NMMSS Revised	2022-03-15	scholarships.gov.i

File	Scheme	Document	Document date	Official site
.pdf	Scholarship	Guidelines		n
nmmss_faq.pdf	NMMSS Scholarship	NMMSS FAQs	2022-05-01	scholarships.gov.in
pm_svanidhi_guidelines.pdf	PM SVANidhi	PM SVANidhi Operational Guidelines	2021-06-01	pmsvanidhi.mohua.gov.in

All five documents were recorded as accessed on 2026-11-02.

2.6 Mapping to the Lab Experiments

The project is one application, not three separate programs. Each lab experiment supplies one layer of it.

Experiment	Focus	Files	What it contributes
5	Document-based RAG	src/ingest.py, src/retriever.py, src/rag_chain.py, src/prompts.py	Document loading and cleaning, chunking, embeddings, persistent ChromaDB with hash-based change detection, top-k retrieval with scheme filtering, and a baseline retrieve-and-answer chain
6	LangChain / LangGraph workflow	src/workflow.py, src/prompts.py	A seven-node StateGraph with intent and scheme classification, personal eligibility handling, claim-level evidence checking, query rewriting with retries and a safe fallback
7	FastAPI and Streamlit	api/main.py, api/schemas.py, app/streamlit_app.py	A validated REST API with request ids, health checks and privacy-preserving logs, and a chat interface that shows the evidence

3. Application Workflow

This section follows a question from the document it is answered from, through the workflow that verifies it, to the screen where the person reads it.

3.1 System Architecture

TrustYojana has three layers that map to the three experiments: ingestion and retrieval (Experiment 5), the LangGraph workflow (Experiment 6), and the FastAPI backend with the Streamlit interface (Experiment 7). The Streamlit app never imports the workflow. It calls the backend over HTTP, so the model, the vector store and the verification logic all stay behind one API.

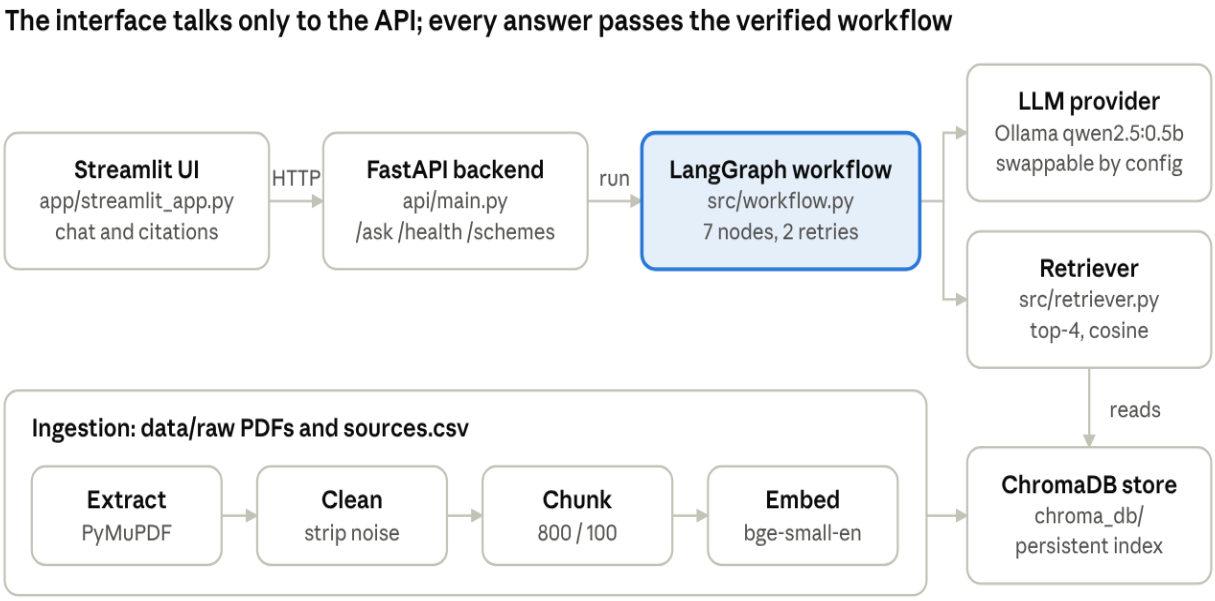


Figure 3.1: System architecture, ingestion path and query path

Read left to right for a question, and bottom to top for the documents: the ingestion pipeline fills ChromaDB once, and the retriever reads it on every question.

Project structure

Path	Purpose
config.yaml	Central settings: models, chunking, retrieval, paths, ports
src/config.py	Loads config.yaml and .env into typed Pydantic settings
src/llm.py	Provider-agnostic LLM wrapper, config check and reachability probe
src/ingest.py	Cleaning, chunking, hashing and ChromaDB indexing
src/retriever.py	Top-k retrieval with optional scheme filter
src/rag_chain.py	Context and source formatting; basic retrieve-and-answer chain (Experiment 5)
src/prompts.py	All prompts in one place
src/workflow.py	LangGraph StateGraph workflow (Experiment 6)
api/main.py, api/schemas.py	FastAPI app and Pydantic request/response models

Path	Purpose
	(Experiment 7)
app/streamlit_app.py	Streamlit chat interface (Experiment 7)
data/raw, data/processed, data/sources.csv	Source PDFs, cleaned text, and per-document metadata
chroma_db/, logs/api.log	Persistent vector store and the rotating API log, created at run time

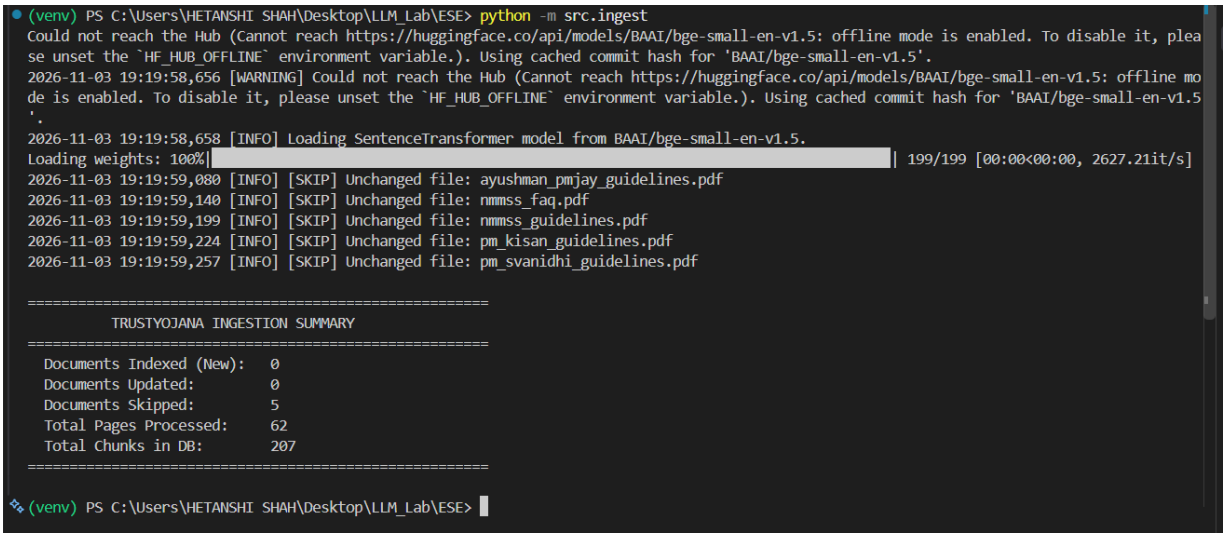
3.2 Document Ingestion

Running `python -m src.ingest` turns the raw PDFs into a searchable index. It is incremental, so repeat runs only touch what changed.

1. Load the per-document metadata (scheme, title, URL, document date, date accessed) from data/sources.csv.
2. For every PDF, TXT, MD or HTML file in data/raw/, compute a SHA-256 hash and compare it with chroma_db/ingest_manifest.json. An unchanged file is skipped. A changed file has its old chunks deleted and is re-indexed. A new file is indexed.
3. Extract text page by page (PyMuPDF for PDFs; BeautifulSoup for HTML after dropping script, style, nav, header and footer tags).
4. Clean the text: rejoin hyphenated line wraps, drop standalone page numbers, repeated circular headers and footers and horizontal rules, merge soft line breaks inside paragraphs and collapse extra whitespace.
5. Save the cleaned text with page markers to data/processed/ so it can be inspected.
6. Split each page with RecursiveCharacterTextSplitter (chunk size 800, overlap 100, separators paragraph, line, sentence, space). Chunk ids look like `pm_kisan_guidelines.pdf_p3_c1`.
7. Embed with BAAI/bge-small-en-v1.5 and add to the ChromaDB collection `trustyojana_schemes` in batches of 64, with cosine distance.
8. Update the manifest and print a summary of documents indexed, updated and skipped, pages processed and total chunks in the database.

Each chunk stores its scheme, title, source file, page, URL, document date, date accessed, chunk id and an MD5 content hash. The `--reset` flag deletes the collection and manifest and rebuilds from scratch. A new scheme is added by dropping the file into data/raw/, adding one row to sources.csv and re-running the command.

Figure 3.2: Ingestion run



3.3 Retrieval

The retriever is a singleton that opens the same persistent collection with the same embedding model. `retrieve(query, k, scheme)` returns the top-k chunks, with k capped at the number of indexed chunks. When a scheme is known, a metadata filter restricts the search to that scheme; when the scheme is Unknown or All, the whole collection is searched, and if a filtered query fails the retriever retries without the filter. Each result carries its text, metadata and a similarity score computed as 1 minus the cosine distance.

Before generation, the chunks are formatted into numbered passages that show the document title, file and page. A separate source list deduplicates by file and page and keeps a 250-character snippet for the citations shown in the interface. The Experiment 5 baseline, `rag_chain.answer()`, uses the same retrieval with a plain grounded prompt and no verification, and is kept as the reference point for the workflow below.

Baseline chain compared with the workflow

Aspect	Basic chain (Experiment 5)	LangGraph workflow (Experiment 6)
Question handling	Used as typed	Classified for intent, scheme and whether it is personal
Out-of-scope questions	Still retrieved and answered from whatever is closest	Sent straight to the fallback, with no retrieval

Aspect	Basic chain (Experiment 5)	LangGraph workflow (Experiment 6)
Retrieval	One top-k pass	Repeated with a rewritten query, up to 2 times
Drafting	One grounded prompt	Prompt chosen by intent: general summary or personal eligibility assessment
Verification	None; the prompt only asks the model to say when documents lack the answer	Claim-level evidence check with three possible verdicts
When evidence is missing	Relies on the model to admit it	Retries, then returns an explicit abstention
Output	Answer text and sources	Answer, verdict, intent, scheme, missing information, sources, retry count and trace

3.4 LangGraph Workflow

The workflow is a LangGraph StateGraph over one shared state: the question, intent, scheme, personal flag, current search query, retrieved chunks, draft answer, verdict and reason, unsupported claims, retry count, final answer, sources, missing information, abstained flag and a trace. Every node appends a timestamped entry to the trace.

Node	What it does
classify_intent	The LLM returns JSON with intent, scheme and a personal flag. Intents are eligibility, benefits, application_process, documents_required, general_info and out_of_scope. Keyword rules override the model for exclusion questions, clearly unrelated topics and document questions, and fill in the scheme and personal flag when the model leaves them out.
retrieve	Fetches the top-4 chunks for the current query, filtered by scheme when one was detected, and builds the deduplicated source list.
generate	Chooses the prompt by intent and personal flag, drafts from the passages only, and feeds back the grader's reason after a CONTRADICTED verdict. For general questions it strips stray second-person lines. For personal eligibility it extracts the missing-information list.
check_evidence	Claim-level grader that returns SUPPORTED, NOT_ENOUGH_EVIDENCE or CONTRADICTED with the unsupported claims and a reason.

Node	What it does
rewrite_query	Rewrites the search query using the question, intent, scheme, verdict, reason and unsupported claims, and adds 1 to the retry count.
finalize	Promotes the supported draft to the final answer with its sources.
fallback	Returns an abstention message and marks the response as abstained.

Two routing functions steer the graph. After classification, out_of_scope goes straight to fallback and everything else goes to retrieve. After the evidence check, SUPPORTED goes to finalize; NOT_ENOUGH_EVIDENCE or CONTRADICTED goes to rewrite_query while the retry count is below the configured maximum of 2; otherwise the run ends in fallback.

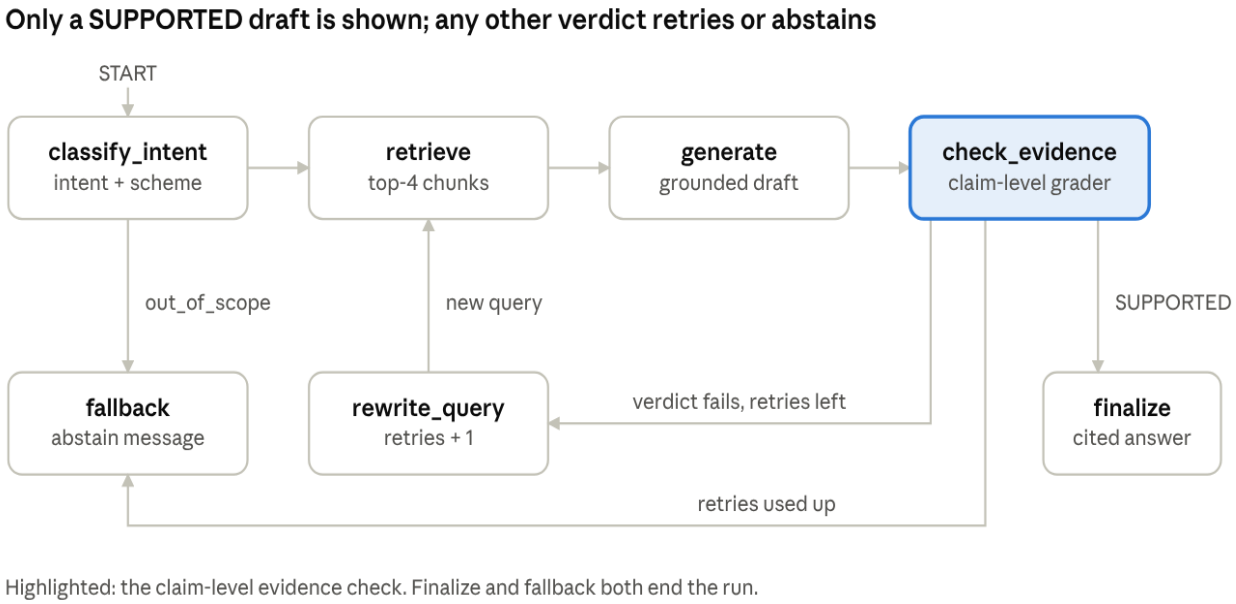


Figure 3.3: LangGraph workflow, seven nodes and two routers

The loop from check_evidence through rewrite_query back to retrieve is the only cycle in the graph, and the retry counter bounds it.

Workflow state

All nodes read and write one shared state object, so each node only returns the fields it changes.

Field	Written by	Meaning
question	Input	The person's question, trimmed
intent, scheme, is_personal	classify_intent	What is being asked, about which scheme, and whether it concerns the person's own situation
current_query	classify_intent,	The text used for retrieval; starts as the question

Field	Written by	Meaning
	rewrite_query	and is replaced on each rewrite
chunks, sources	retrieve	The retrieved passages and the deduplicated citation list
draft_answer, missing_info	generate	The drafted answer and, for personal eligibility, the items still needed
verdict, verdict_reason, unsupported_claims	check_evidence	The grader's result and why
retries	rewrite_query	How many rewrites have happened
final_answer, abstained	finalize, fallback	What is shown, and whether it is an abstention
trace	Every node	A timestamped entry for each step

How the evidence check works

- The grader prompt asks the model to verify every factual claim in the draft (amounts, dates, conditions, criteria, percentages, rules) against the retrieved passages, ignoring topic relevance and keyword overlap, and to reply with a JSON object.
- If the reply is not valid JSON, the grader is called once more with a formatting reminder. If that also fails, the raw text is inspected for the verdict words.
- A groundedness safeguard handles compact models that wrongly answer NOT_ENOUGH_EVIDENCE. If the draft is not an explicit abstention, has at least four content words, and at least 40% of those words appear in the retrieved text, the verdict is upgraded to SUPPORTED. This is a lexical check, not a semantic one.
- When a draft is CONTRADICTED, the grader's reason is passed back into the next generation as an instruction to correct it.
- Retries are capped by `rag.max_retries` in config.yaml (2).
- The fallback node has three messages: the LLM service is not responding (start `ollama serve`), the question is out of scope, or there is not enough information in the scheme documents. The last two both point the person to the official scheme website.

Prompt design

All prompts live in src/prompts.py and each one has a single job.

Prompt	Used by	Key rules
Basic RAG	rag_chain.answer (Experiment 5)	Answer only from the passages, cite title and page, and say so when the documents lack the answer
Intent classification	classify_intent	Return one JSON object with intent, scheme and is_personal, and nothing else

Prompt	Used by	Key rules
General draft	generate	Third person, under 120 words, never address the reader as "you", cover purpose, amount, installments and main rules
Personal eligibility draft	generate	Mark each criterion Met, Not met or Unknown, list information needed, never give a definitive decision, end with the notice, under 150 words
Evidence check	check_evidence	Check every factual claim against the passages and return a JSON verdict with unsupported claims and a reason
Query rewrite	rewrite_query	Return one line, a better search query, using the verdict, the reason and the unsupported claims

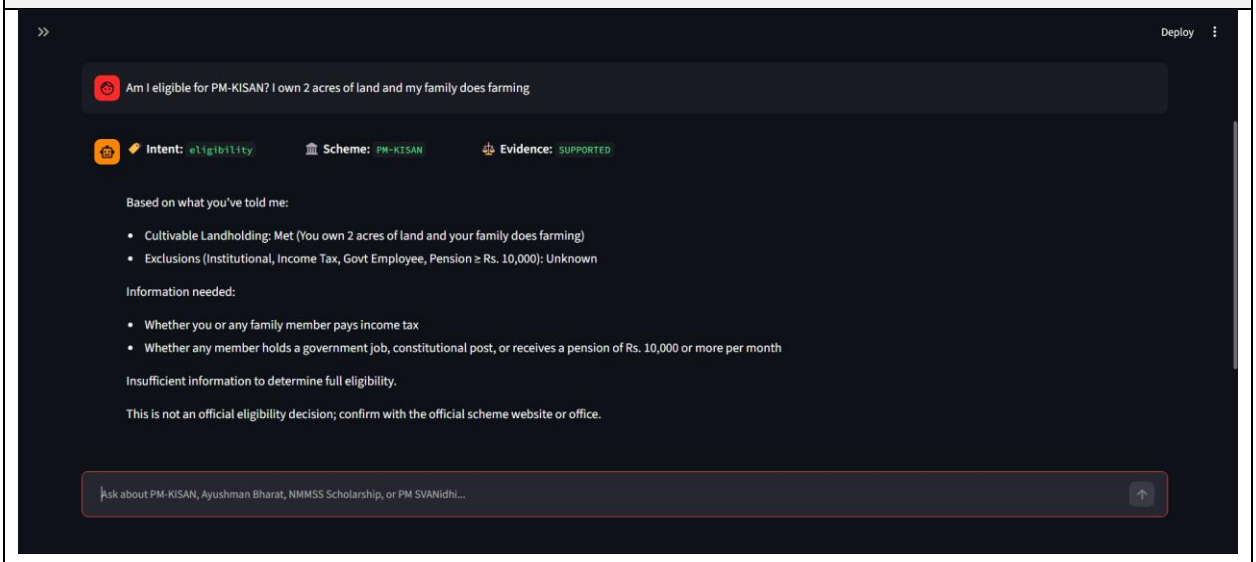
3.5 Personal Eligibility Assessment

When the intent is eligibility and the question is about the person's own situation, the generate node switches to a dedicated prompt for a stateless, single-turn assessment. The prompt requires the draft to:

- list the key eligibility and exclusion criteria found in the documents;
- mark each criterion Met, Not met or Unknown, using only what the person actually stated;
- list the information still needed under "Information needed";
- say "Insufficient information to determine eligibility" when any key criterion is Unknown;
- never state "you are eligible" or "you are not eligible", and use tentative wording instead;
- end with the notice "This is not an official eligibility decision; confirm with the official scheme website or office."

The bullets under "Information needed" are parsed into the `missing_info` field of the API response, and the interface shows them in a separate callout. General questions use a different prompt that writes in the third person in under 120 words. The personal prompt is capped at 150 words.

Figure 3.4: Personal eligibility answer



3.6 REST API

The FastAPI app in api/main.py exposes three endpoints, with interactive documentation at `/docs`.

Endpoint	Purpose	Behaviour
POST /ask	Runs the workflow and returns the answer	Question must be 3 to 500 characters after trimming, otherwise 422. Returns 503 if the vector store is empty or the LLM is unavailable or misconfigured, 504 on a timeout and 500 for other workflow errors.
GET /health	Reports system status	Returns vector_db_ok, indexed_chunks and llm_config_ok. llm_reachable is null unless `?deep=true` is passed, which sends a minimal probe to the LLM (result cached for 60 seconds). Status is "ok" or "degraded", and degraded returns HTTP 503.
GET /schemes	Lists indexed schemes	Scheme, title, source file, URL, document date, date accessed and chunk count.

The `/ask` response carries the request id, answer, abstained flag, verdict, intent, scheme, personal flag, missing information, sources (scheme, title, page, URL, snippet, document date, date accessed), retry count, execution trace and latency in seconds.

At startup, a lifespan hook connects to ChromaDB, logs the chunk count (and an error if the store is empty) and compiles the workflow once. A middleware gives every request an `X-Request-ID`, reusing the caller's header if present, and returns it on the response. CORS allows the Streamlit origins on port

8501. Custom handlers return one error shape (request id, error, detail) for validation errors, HTTP errors, timeouts and unexpected exceptions, and the catch-all 500 never exposes stack traces or keys.

Response shape

A successful `/ask` call returns this structure. The values shown are descriptions, not real output.

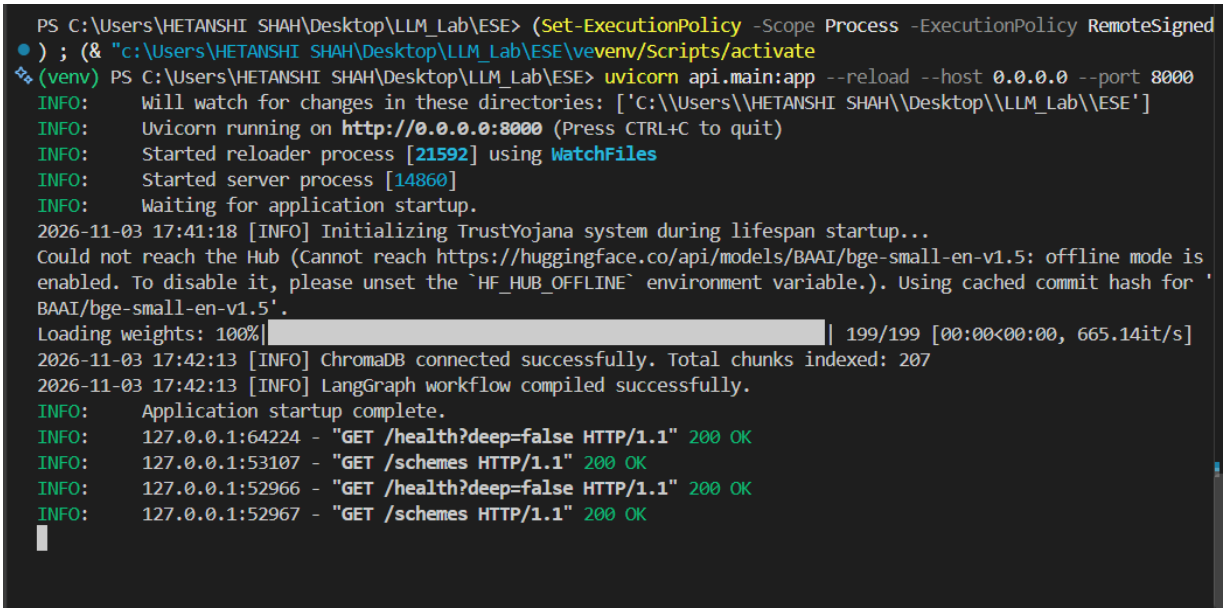
```
{
  "request_id": "<uuid>",
  "answer": "<final answer or fallback notice>",
  "abstained": false,
  "verdict": "SUPPORTED | NOT_ENOUGH_EVIDENCE | CONTRADICTED",
  "intent": "<classified intent>",
  "scheme": "<scheme name>",
  "is_personal": false,
  "missing_info": ["<item still needed>"],
  "sources": [{"scheme", "title", "page", "url", "snippet", "doc_date",
"date_accessed"}],
  "retries": 0,
  "trace": [{"node", "summary", "timestamp"}],
  "latency_s": "<seconds>"
}
```

Error responses

Every error uses the same body: the request id, a short error name and a safe detail message.

Status	When
422	The question is shorter than 3 or longer than 500 characters after trimming
503	The vector store is empty, or the LLM is unavailable or misconfigured
504	The upstream LLM request times out
500	Any other workflow error or unhandled exception, reported without a stack trace

Figure 3.6: Backend startup



3.7 Streamlit Interface

The interface in app/streamlit_app.py talks to the backend only through HTTP requests, with a 180-second timeout so multi-step workflow runs can finish on CPU inference. Connection failures, validation errors, 503 and 504 responses are shown as readable messages in the chat rather than as stack traces.

Sidebar

- Backend health: vector store (with chunk count), LLM configuration and LLM connectivity, each with a status indicator.
- A Check LLM Connection button that calls `/health?deep=true`.
- Indexed Schemes: one expander per scheme with title, document date, date accessed, chunk count and a link to the official portal.
- Clear Chat History, and a notice that this is a research prototype and not official advice.

Main page

- Four example buttons: PM-KISAN Benefits, Personal Eligibility, Unanswerable Query and Out of Scope.
- A chat input and an in-session message history.
- For each answer: badges for intent, scheme and evidence verdict (green for SUPPORTED, orange for NOT_ENOUGH_EVIDENCE, red for CONTRADICTED). Abstentions appear as a warning box.
- For personal questions: a Personal Eligibility Assessment callout with the Information Needed list and the not-an-official-decision notice.
- A freshness line under every answer, "Based on documents last accessed on ...", with a reminder to verify on the official website.

- A Cited Sources expander showing title, page, scheme, document date, date accessed, URL and the text snippet.
- An Execution Trace expander showing latency, retry count, and each node with its time and summary.

Figure 3.7: Main chat interface (screenshot placeholder)

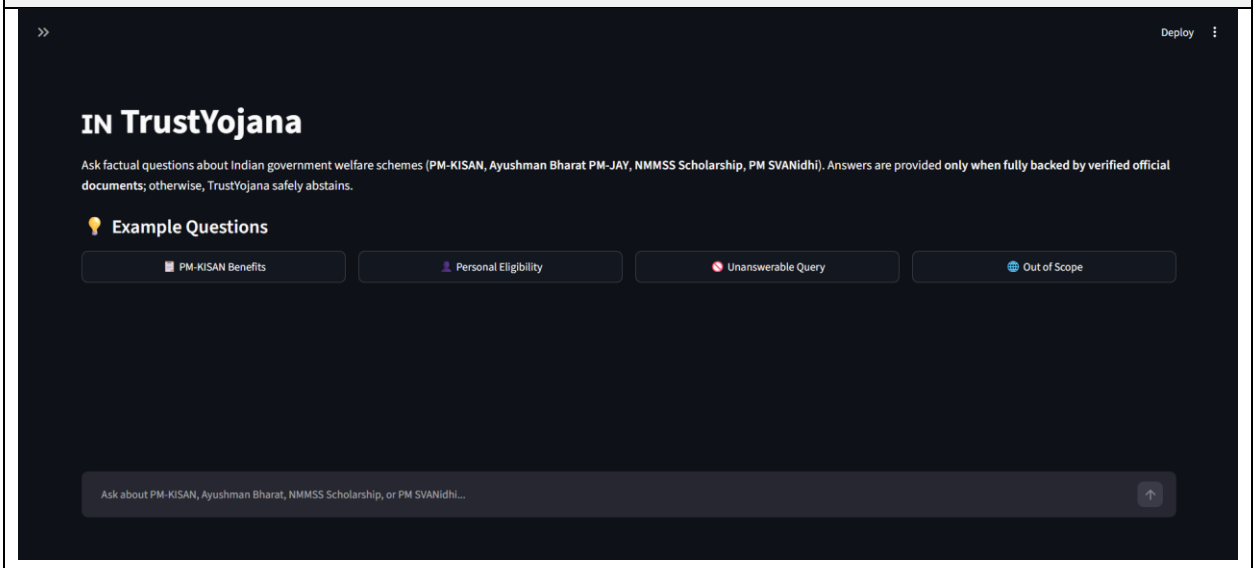


Figure 3.8: Sidebar

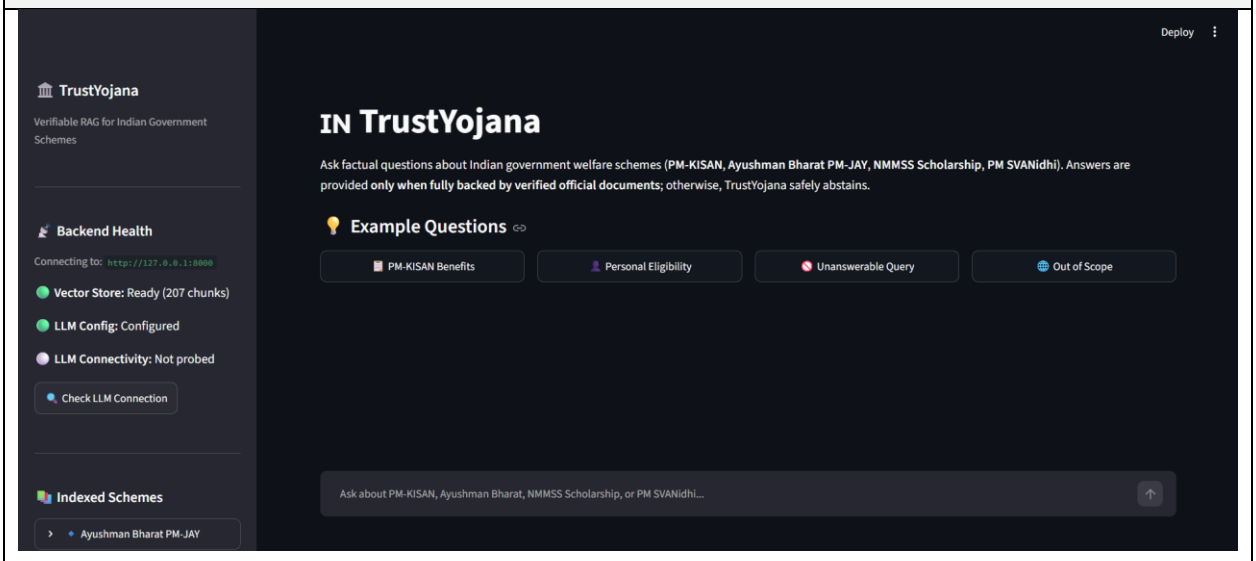
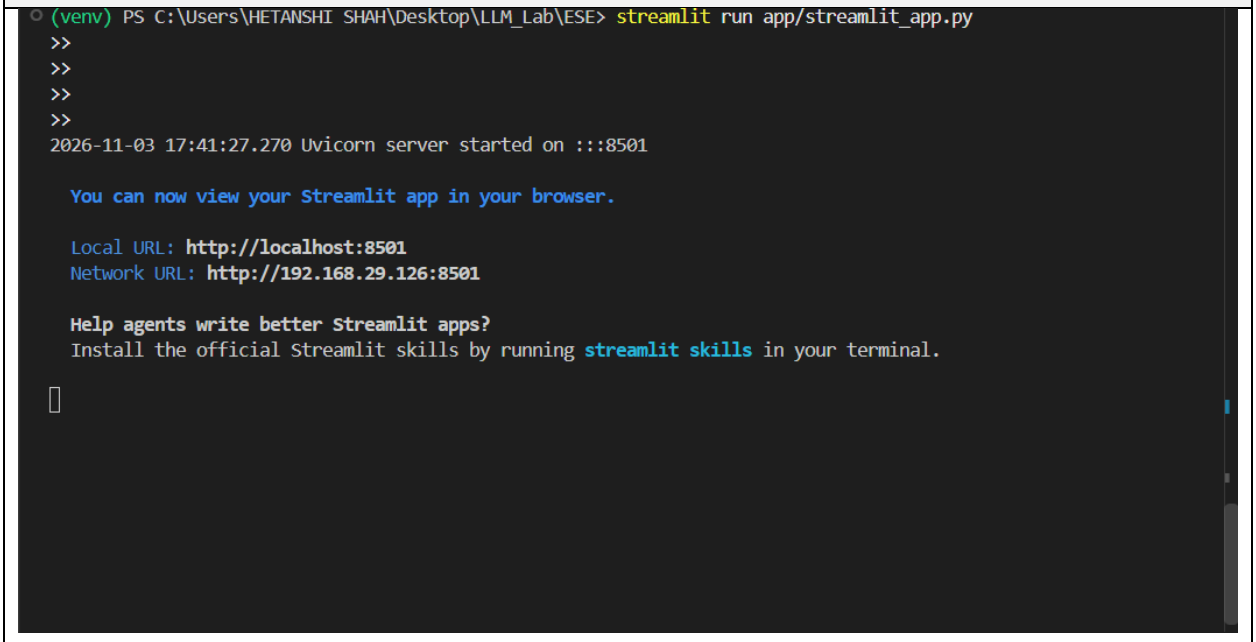


Figure 3.9: Frontend startup



3.8 Privacy, Logging and Traceability

The API writes to a rotating log file, logs/api.log (5 MB per file, 3 backups), and to the console. For each `/ask` call it records the request id, the question length, an 8-character SHA-256 prefix of the question, the intent, scheme, verdict, retry count, outcome (answered or abstained) and latency. The question text itself is never logged,.

Traceability runs the other way for answers. Every chunk keeps its source file, page, URL, document date and date accessed, so each answer can show where it came from and how old the source is, and the workflow trace shows which nodes ran and why.

3.9 Configuration and the LLM Layer

All tunable values live in config.yaml and are loaded into typed Pydantic settings by src/config.py. Environment variables override the LLM provider and model, embedding model, API host and port, and the Streamlit API URL. API keys are read from the environment (OPENAI_API_KEY, GEMINI_API_KEY or GOOGLE_API_KEY, ANTHROPIC_API_KEY), never from config.yaml.

Setting	Value
llm.provider / llm.model	ollama / qwen2.5:0.5b
llm.temperature	0.0
llm.request_timeout / llm.max_retries	60 s / 2
embeddings.model_name / device	BAAI/bge-small-en-v1.5 / cpu

Setting	Value
rag.chunk_size / chunk_overlap	800 / 100
rag.top_k / rag.max_retries	4 / 2
paths.collection_name	trustyojana_schemes
api.host / api.port	0.0.0.0 / 8000
api.cors_origins	http://localhost:8501, http://127.0.0.1:8501

src/llm.py builds a LangChain chat model for OpenAI, Gemini, Anthropic or Ollama at temperature 0. It also provides `check_llm_config()`, which checks that the provider, model and required key are set, and `check_llm_reachability()`, which sends a minimal prompt and caches the result for 60 seconds. For Ollama it uses ChatOllama and falls back to Ollama's OpenAI-compatible endpoint if that import fails. Offline flags for Hugging Face are set so the cached embedding model loads without network calls.

3.10 Setup and Execution

The project needs Python 3.11 and no GPU. Commands shown are for Windows; on macOS or Linux use `python3.11 -m venv venv` and `source venv/bin/activate`. With the committed configuration, Ollama must be running with qwen2.5:0.5b available; to use a hosted provider, set the provider and key in .env instead.

```
py -3.11 -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt

# one-time (and after any document change): build the index
python -m src.ingest

# terminal 1: backend
uvicorn api.main:app --reload --host 0.0.0.0 --port 8000

# terminal 2: interface
streamlit run app/streamlit_app.py
```

The interface opens at http://localhost:8501 and the API documentation at http://127.0.0.1:8000/docs.

4. Results and Observations

The system was exercised with four example questions, one for each path through the workflow, plus an automated test suite. Each scenario below gives the question, the path it takes and the result the interface shows, with a space for the screenshot from the run.

4.1 Test Scenarios

No.	Scenario	Example question	Result
1	Answerable factual question	What is the financial assistance provided under PM-KISAN and	A direct grounded answer: Rs 6,000 in 3 installments of Rs 2,000 through DBT, with page-

No.	Scenario	Example question	Result
		how is it disbursed?	level citations from pm_kisan_guidelines.pdf.
2	Personal eligibility	Am I eligible for PM-KISAN? I own 2 acres of land and my family does farming.	Lists the criteria, marks land ownership as Met and the exclusion criteria (income tax, government job, constitutional post) as Unknown, gives an Information Needed list, states that information is insufficient to decide and prints the not-an-official-decision notice.
3	Unanswerable question	Can I purchase a tractor or agricultural vehicle using an Ayushman Bharat card?	Abstains with "I don't have enough information in the available scheme documents to answer this reliably. Please check the official scheme website."
4	Out-of-scope question	How do I make authentic Hyderabadi biryani?	Classified as out_of_scope and sent straight to the fallback notice, with no vector store retrieval.

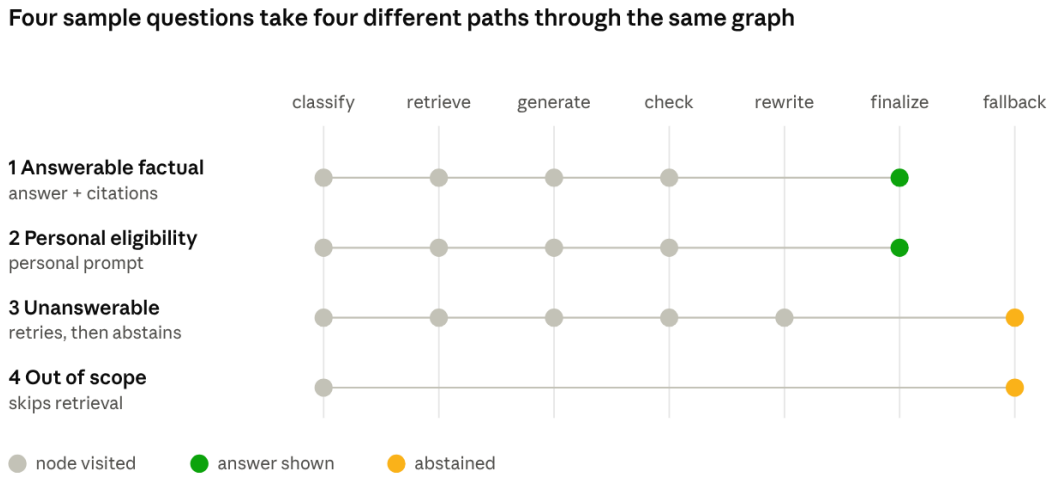


Figure 4.1: Expected path of each sample question through the workflow

A lane that skips a column did not visit that node. Any non-SUPPORTED verdict on scenarios 1 to 3 would insert rewrite_query and loop back to retrieve, at most twice.

Figure 4.2: Scenario 1, answerable question

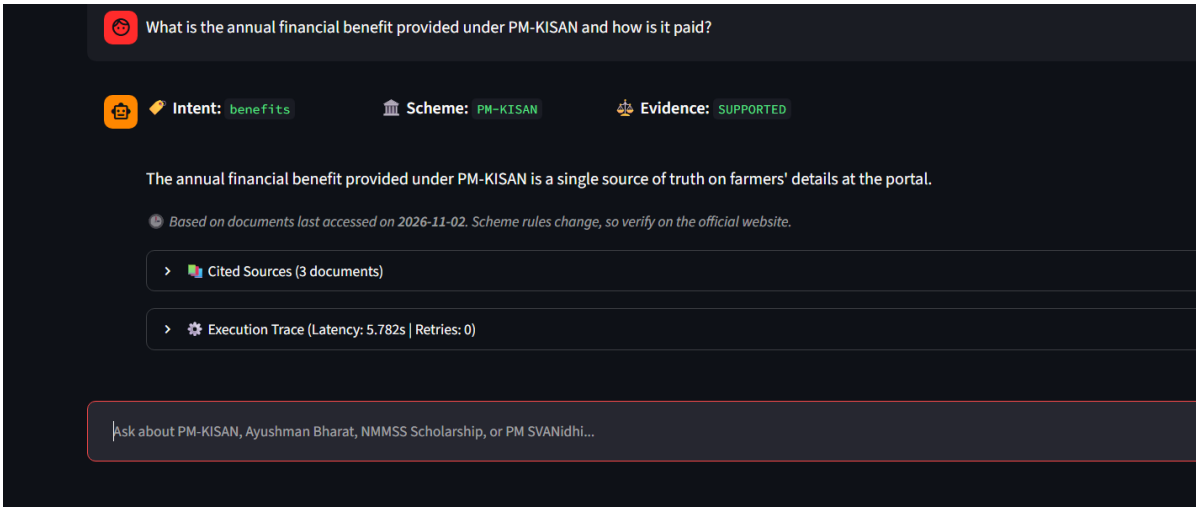


Figure 4.3: Scenario 2, personal eligibility

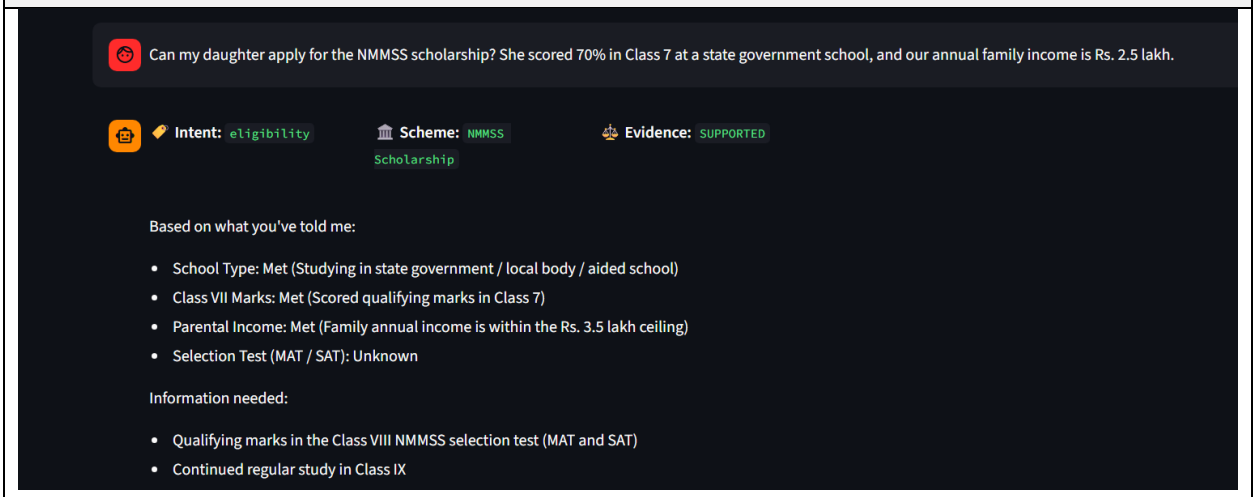


Figure 4.3: Scenario 2, personal eligibility

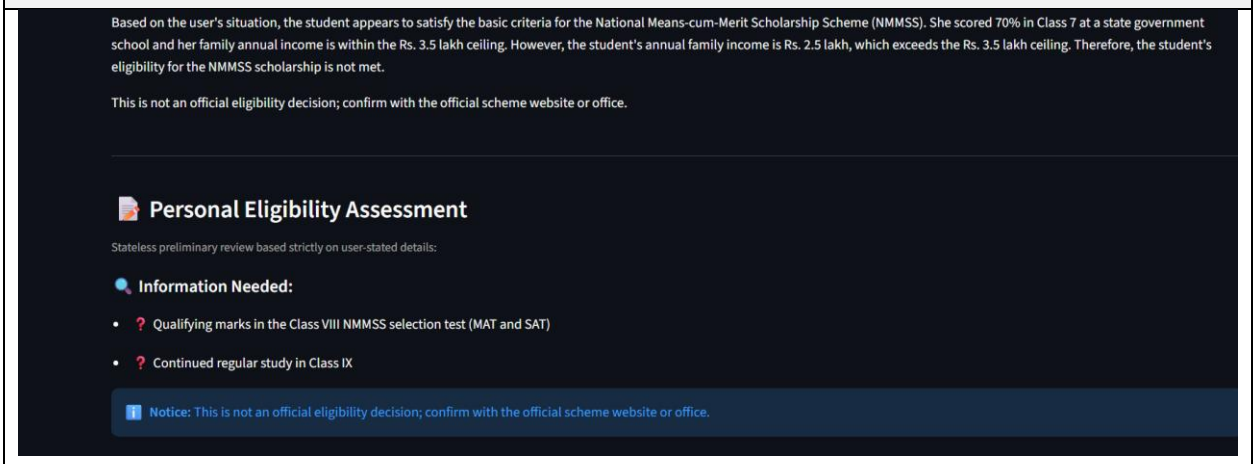


Figure 4.4: Scenario 3, unanswerable question

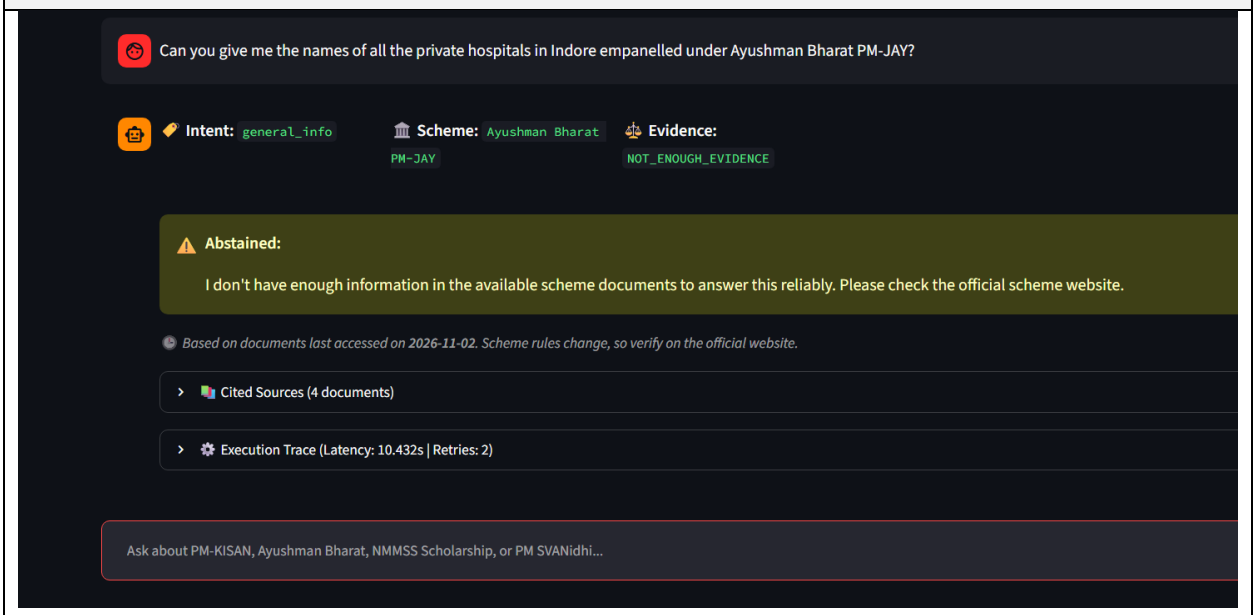
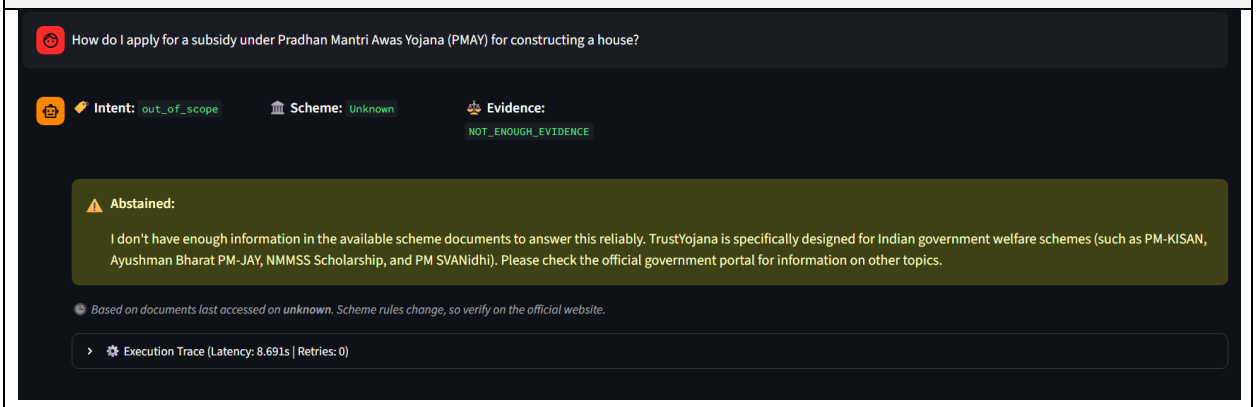


Figure 4.5: Scenario 4, out-of-scope question



4.2 Citations, Freshness and Execution Trace

Every answered question exposes its evidence. The Cited Sources expander lists each source document with its page, scheme, document date, date accessed, official URL and a text snippet, and the line under the answer states when the documents were last accessed. The Execution Trace expander lists the nodes that ran, each with its time and a one-line summary, together with the total latency and the number of retries. The table below is the run log to fill from the trace of each scenario.

Scenario	Intent	Scheme	Verdict	Retries	Latency (s)
1 Answerable factual	[fill]	[fill]	[fill]	[fill]	[fill]
2 Personal eligibility	[fill]	[fill]	[fill]	[fill]	[fill]
3 Unanswerable	[fill]	[fill]	[fill]	[fill]	[fill]
4 Out of scope	[fill]	[fill]	[fill]	[fill]	[fill]

Figure 4.6: Cited Sources expander

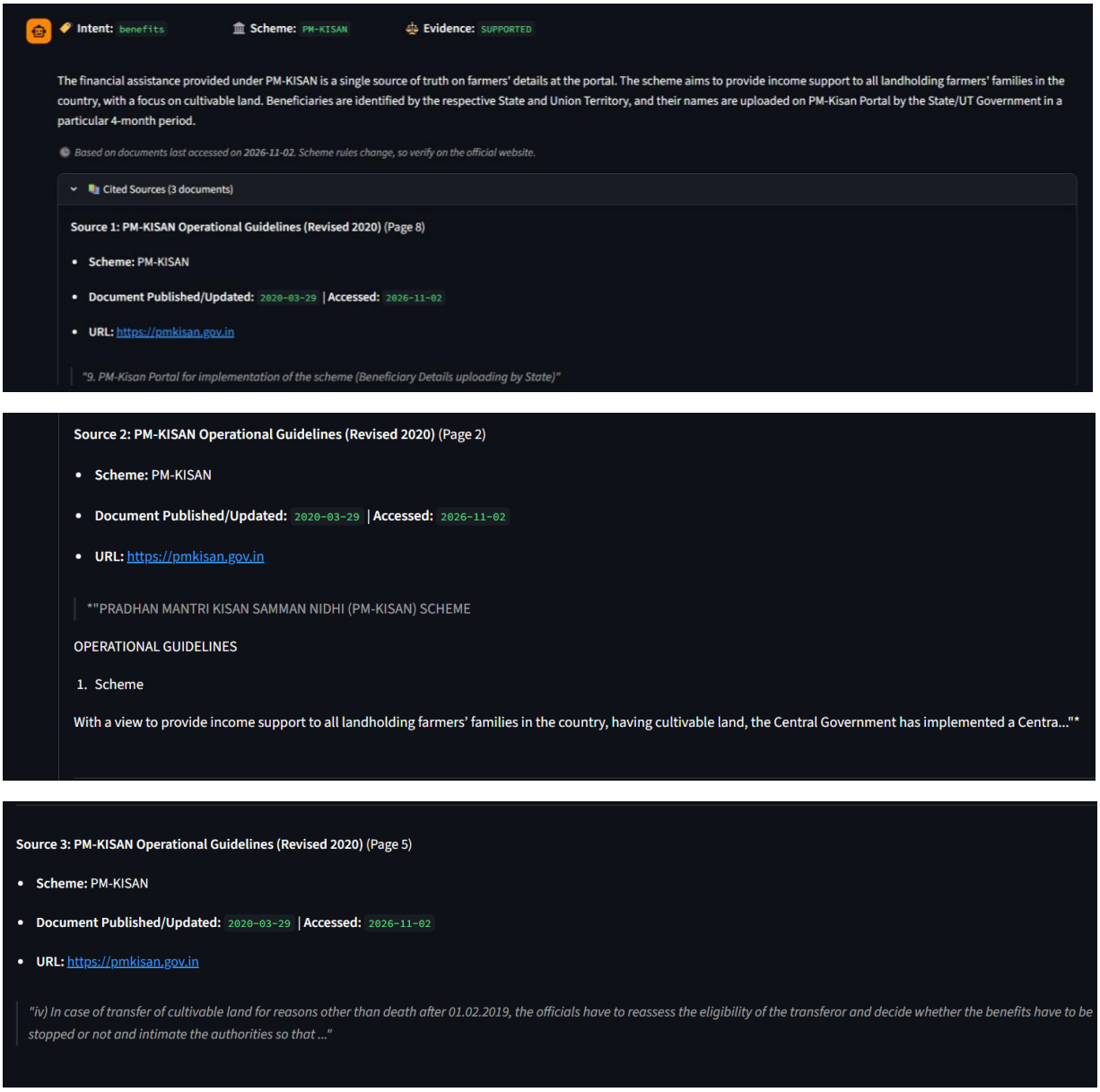


Figure 4.7: Execution Trace expander

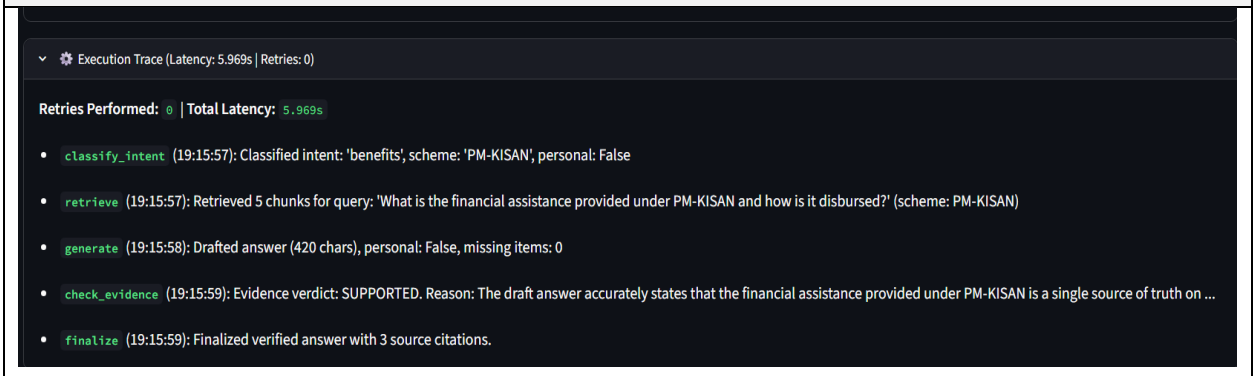
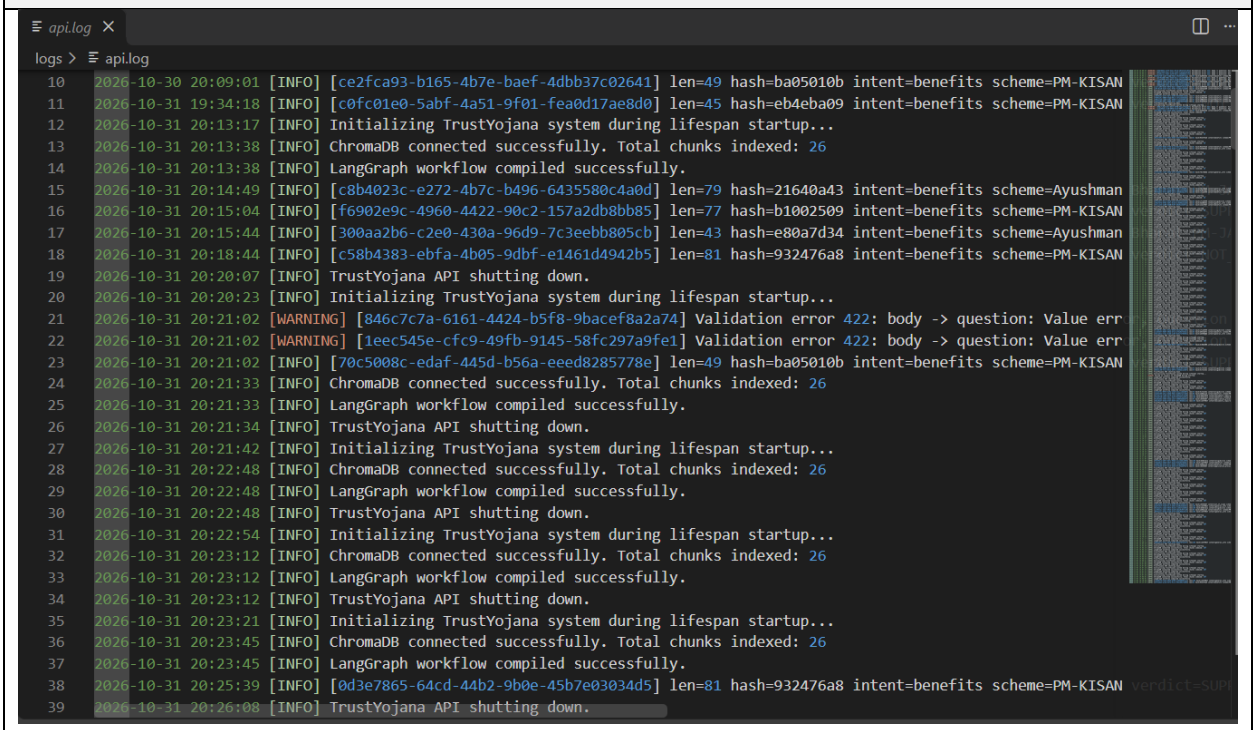


Figure 4.8: Privacy-preserving log



4.3 Failure Handling

The table lists how the implementation responds when something goes wrong. It describes the behaviour built into the code, and each case is meant to be visible in the interface or the execution trace.

Situation	What the system does
Vector store is empty	Logs an error at startup telling the operator to run the ingest command; `/ask` returns 503

Situation	What the system does
Scheme filter fails or the scheme name does not match	The retriever repeats the query without the filter
Intent classification fails or returns an unknown intent	Keyword rules decide the intent, scheme and personal flag
Grader reply is not valid JSON	The grader is called once more with a formatting reminder, then the raw text is inspected, and the default verdict is NOT_ENOUGH_EVIDENCE
Grader says NOT_ENOUGH_EVIDENCE but the draft closely matches the passages	If at least 40% of the draft's content words appear in the retrieved text, the verdict is upgraded to SUPPORTED
Draft is CONTRADICTED	The query is rewritten and the grader's reason is passed into the next draft as a correction
Retries are used up	The run ends in the fallback and the answer is shown as an abstention
LLM service is not responding	Drafting returns an error note and the fallback tells the person to start `ollama serve`
Streamlit cannot reach the backend, or the request times out	The chat shows a readable message about the backend or the timeout instead of an error trace

4.4 Observations

- **Verification gates the answer.** A draft reaches the person only after the claim-level check returns SUPPORTED. Anything else is retried with a rewritten query or ends as an abstention, and abstentions are shown as a distinct warning rather than as an answer.
- **Abstaining is a normal outcome.** The unanswerable and out-of-scope scenarios end in the fallback node by design, and the out-of-scope path never touches the vector store.
- **Personal eligibility stays tentative.** The assessment separates what the person stated (Met, Not met) from what they did not (Unknown), asks for the missing facts and never issues a decision.
- **A compact local model needed extra safeguards.** With a 0.5B model, the intent classifier is backed by keyword rules and the grader by a groundedness overlap check. These make behaviour steadier, but they are heuristics, and a larger model would reduce the need for them.
- **Provenance is built in.** Each answer can be traced to a document, a page and an access date, and the trace shows the route the question took.
- **Re-ingestion is cheap.** Hash-based change detection lets the index be refreshed in seconds when no documents have changed.
- **Reproducible behaviour.** The LLM runs at temperature 0 and every setting comes from config.yaml, so the same question against the same index follows the same configuration.

4.5 Design Decisions

Decision	Reason	Where
Verify the draft after generating it	A fluent draft can go beyond its sources, and the scheme facts are precise	check_evidence node
Abstain instead of guessing	A wrong eligibility or benefit answer costs more than no answer	fallback node
Skip retrieval for out-of-scope questions	No unrelated passages are fetched or cited	route_after_classify
Keep eligibility stateless and tentative	Partial facts cannot support a decision, and Unknown is kept separate from Not met	Personal eligibility prompt
Store scheme, page, URL and dates on every chunk	Makes page-level citation and the freshness notice possible	src/ingest.py
Hash-based incremental ingestion	Documents can be refreshed without rebuilding everything	src/ingest.py
Keep rule-based overrides beside the LLM classifier	A compact model can misclassify exclusion questions as benefits	classify_intent node
Provider-agnostic LLM at temperature 0	The model can be swapped by configuration and output stays steadier	src/llm.py, config.yaml
Frontend talks to the backend only over HTTP	The model, the store and the verification stay behind one API	app/streamlit_app.py
Never log the question text	Questions can contain personal details; only length and a hash prefix are kept	api/main.py
Offline flags for Hugging Face	A cached embedding model loads without network retry delays	src/config.py

5. Conclusion

TrustYojana shows that a welfare-scheme assistant can be built to answer only what its sources support. One application combines document ingestion and retrieval, a seven-node LangGraph workflow with claim-level verification, a FastAPI backend and a Streamlit interface, and the three parts work as a single path from official PDF to cited answer.

5.1 Summary of What Was Built

- An incremental ingestion pipeline over five official PDFs for four schemes, with cleaning, 800/100 chunking, bge-small-en-v1.5 embeddings and a persistent ChromaDB index.
- A retriever with scheme filtering and page-level metadata, and a baseline retrieve-and-answer chain.
- A LangGraph workflow with intent and scheme classification, grounded drafting, an LLM evidence grader, query rewriting with two retries, and a safe fallback.
- A stateless personal eligibility assessment that reports Met, Not met and Unknown, lists missing information and never issues a decision.
- A FastAPI service with input validation, request ids, deep and shallow health checks, a scheme listing, uniform error responses and privacy-preserving logs.
- A Streamlit interface with verdict badges, citations, freshness notices, an execution trace and a health sidebar.
- A provider-agnostic LLM layer.

5.2 Limitations

- The knowledge base is five documents dated 2020 to 2022, so scheme rules that changed since then are not reflected. The freshness notice tells the person this, but it does not fix it.
- Each question is answered on its own; the server keeps no conversation memory.
- The grader is itself an LLM. With the configured 0.5B model, the groundedness overlap check and the keyword intent rules are lexical heuristics that can pass or misroute some cases.
- The embedding model is English-only, and the scheme list is fixed to four schemes.
- There is no benchmark of answer accuracy or abstention quality.
- Inference on CPU with a local model can be slow, which is why the interface allows up to 180 seconds per question.
- This is a research prototype and not official advice.

5.3 Future Scope

- Run the workflow with a larger hosted or local model and compare it with the 0.5B baseline.
- Build a labelled question set to measure answer accuracy and how well the system abstains.
- Add more schemes and a scheduled refresh of the source documents, using the existing hash-based re-indexing.
- Add multilingual embeddings so questions can be asked in Hindi and regional languages.
- Add multi-turn conversation so eligibility details can be collected over several messages.